

Introduction to Software Packages

Introduction to Software Packages

1. spaCy

- spaCy is an open-source software library for advanced natural language processing in Python.
- It is designed specifically for production use and helps build applications that process large volumes of text.
- Key features include tokenization, part-of-speech tagging, dependency parsing, named entity recognition, and word vectors.

python

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("I love Natural Language Processing.")
for token in doc:
    print(token.text, token.pos_)
```

Introduction to Software Packages

2.NLTK (Natural Language Toolkit)

- NLTK is a leading platform for building Python programs to work with human language data.
- It provides easy-to-use interfaces to over 50 corpora and lexical resources, along with text-processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning.

```
python
```

```
import nltk  
from nltk.tokenize import word_tokenize  
nltk.download('punkt')  
text = "NLTK is a leading platform for building Python programs."  
print(word_tokenize(text))
```

Introduction to Software Packages

3. Gensim

- Gensim is a Python library for topic modeling and document similarity analysis.
- It uses modern statistical machine learning to extract semantic topics from documents.
- It is particularly known for its efficiency in handling large text collections using algorithms like Word2Vec, Doc2Vec, and Latent Dirichlet Allocation (LDA).

python

```
from gensim.models import Word2Vec  
sentences = [["machine", "learning"], ["deep", "learning"]]  
model = Word2Vec(sentences, vector_size=50, min_count=1, workers=4)  
print(model.wv["machine"])
```

Introduction to Software Packages

4. PyTorch

- PyTorch is an open-source machine learning framework based on Torch.
- It provides tools for deep learning and tensor computation with GPU acceleration.
- It is widely used in NLP, computer vision, and generative models due to its flexibility, dynamic computation graph, and strong community support.

python

```
import torch
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2 + 3 * x + 5
y.backward()
print(x.grad)  # derivative of y wrt x
```